

IntelliSpoil: Hybrid Deep Learning for Rapid Detection of Fruits and Vegetables Spoilage Using Autoencoder and CNN

*Coursework Assignment for Artificial Neural Network STW7088CEM

Aishwarya Rai
Softwarica College of IT & E-Commerce
Coventry University
raia39@uni.coventry.ac.uk

Abstract—Food waste constitutes a major global challenge with significant economic and environmental consequences, a large proportion of which arises from inefficient identification of spoilage in fruits and vegetables. This study presents an automated image-based food quality assessment framework for classifying produce as fresh or spoiled using deep learning techniques. The proposed method employs a hybrid architecture that integrates a convolutional autoencoder for unsupervised latent feature extraction with a convolutional neural network for binary classification. Experiments are conducted using publicly available fruit and vegetable image datasets, including Fruits 360 and freshness-labelled datasets from Kaggle [1]–[3]. The proposed system provides a non-destructive and scalable solution with potential applicability in retail and supply chain quality control.

Keywords: Deep Learning, Convolutional Autoencoder, Convolutional Neural Network, Food Quality Assessment, Image Classification

I. INTRODUCTION

Food waste has reached critical levels worldwide, with approximately one-third of all food produced for human consumption lost or wasted annually [5]. This inefficiency contributes to excessive resource utilisation, increased greenhouse gas emissions, and substantial economic losses across the food supply chain. Fruits and vegetables are particularly susceptible to spoilage due to their high moisture content, biological activity, and sensitivity to environmental conditions during storage and transportation.

Current food quality assessment practices rely predominantly on manual visual inspection. While widely adopted, this approach is labour-intensive, subjective, and prone to inconsistency, particularly in large-scale distribution environments. Furthermore, early-stage spoilage often manifests through subtle visual and textural variations that may not be reliably detected through human observation alone.

Recent advances in computer vision and deep learning have enabled automated analysis of visual data for food quality assessment. Convolutional Neural Networks (CNNs) have demonstrated strong performance in image classification tasks by learning hierarchical feature representations [4]. However,

most CNN-based approaches depend heavily on large labelled datasets, which are costly and time-consuming to acquire in practical food inspection scenarios. To address this limitation, this research incorporates unsupervised representation learning through convolutional autoencoders to extract informative latent features, which are subsequently employed for supervised freshness classification.

II. BACKGROUND

A. Challenges in Food Quality Assessment

Perishable agricultural produce undergoes rapid quality degradation, making timely and accurate assessment critical in modern food supply chains. Manual inspection methods are often insufficient for large-scale operations and are influenced by human factors such as fatigue, experience, and environmental conditions. These limitations can result in inconsistent grading and delayed detection of spoilage, ultimately increasing food waste [5].

B. Deep Learning Approaches for Food Freshness Detection

Deep learning approaches, particularly those based on Artificial Neural Networks (ANNs), have been increasingly explored for food freshness and spoilage detection. CNNs are capable of learning discriminative spatial features related to texture, colour variation, and surface degradation directly from image data [4]. Despite their effectiveness, supervised CNN models typically require extensive labelled datasets, which may limit their scalability and real-world deployment.

Unsupervised deep learning methods, such as convolutional autoencoders, provide an alternative by learning compact latent representations without the need for annotated data. These representations capture intrinsic visual characteristics associated with freshness and spoilage. In this work, latent features learned by a convolutional autoencoder are utilised as input to a CNN-based classifier, enabling effective freshness detection while reducing dependence on large labelled datasets.

III. METHODOLOGY

A. Datasets

To ensure diversity and robustness, three publicly available datasets are utilised:

- **Fruits 360 Dataset [1]:** A large-scale dataset containing over 90,000 images across 131 fruit and vegetable categories.
- **Fruit and Vegetable Disease Dataset [2]:** Includes labelled images of healthy and rotten produce.
- **Fresh and Stale Classification Dataset [3]:** Specifically designed for binary freshness classification.

B. Data Preprocessing

Prior to training, all images undergo the following preprocessing steps:

- 1) **Resizing:** Images are resized to 450×450 pixels to maintain spatial detail.
- 2) **Normalization:** Pixel intensities are scaled to the range $[0, 1]$:

$$p_{norm} = \frac{p}{255} \quad (1)$$

- 3) **Label Encoding:** Class labels are encoded as:

- Fresh / Healthy $\rightarrow 0$
- Rotten / Stale $\rightarrow 1$

C. Model Architecture

1) *Convolutional Autoencoder:* The convolutional autoencoder (CAE) is employed to learn compact latent representations of fruit and vegetable images in an unsupervised manner. The encoder accepts an RGB image of size $IMG_SIZE \times IMG_SIZE \times 3$ and progressively reduces its spatial resolution using four convolutional blocks. Each block consists of a 3×3 convolution with stride 2, batch normalization, and ReLU activation. The number of filters increases from 32 to 256 in successive blocks, allowing the network to capture increasingly complex and abstract features at deeper layers while compressing spatial information. The resulting feature maps are flattened and passed through a dense layer to produce a 512-dimensional latent vector, which forms the bottleneck representation.

The decoder reconstructs the input image by mirroring the encoder structure. The latent vector is first expanded through a dense layer and reshaped into a three-dimensional feature volume. A sequence of transposed convolution layers with stride 2, batch normalization, and ReLU activation is then used to progressively upsample the feature maps. The final transposed convolution layer employs a sigmoid activation to constrain pixel values to the range $[0, 1]$, followed by a resizing operation to ensure exact reconstruction of the original image dimensions.

a) *Training Configuration:* The CAE is trained to minimize the Mean Squared Error (MSE) reconstruction loss:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2 \quad (2)$$

Training is performed for 20 epochs using the **Adam optimizer** with a learning rate of 10^{-3} .

TABLE I
CONVOLUTIONAL AUTOENCODER ARCHITECTURE

Component	Configuration	Purpose
Input	$IMG_SIZE \times IMG_SIZE \times 3$	RGB image input
Encoder	Conv blocks (32–256 filters)	Hierarchical feature extraction
Latent Space	Dense (512 units)	Compact representation
Decoder	Conv2DTranspose blocks	Image reconstruction
Output	Sigmoid + Resizing	Valid pixel intensities
Loss Function	Mean Squared Error	Reconstruction objective

2) *CNN-Based Classifier:* The CNN-based classifier architecture leverages transfer learning by utilizing a pre-trained convolutional encoder as a fixed feature extractor. This approach preserves spatial hierarchies learned during the reconstruction phase for binary classification.

a) *Architecture Overview:* The encoder output is processed via Global Average Pooling (GAP) to reduce dimensionality before entering the classification head. The specific architectural details are summarized in Table II.

TABLE II
CLASSIFICATION HEAD ARCHITECTURE

Layer	Type	Output Size	Specifications
Input	Latent Vector ($\mathbf{h}$)	Variable	From Frozen Encoder
Pooling	Global Avg Pool 2D	$1 \times C'$	Spatial Reduction
Dense 1	Fully Connected	256	ReLU, Batch Norm
Regularization	Dropout	256	Rate = 0.5
Dense 2	Fully Connected	64	ReLU
Regularization	Dropout	64	Rate = 0.3
Output	Dense (Sigmoid)	1	Binary Classification

b) *Mathematical Formulation:* The feed-forward process for the classification head, including Bernoulli dropout masks ($\mathbf{m}$) applied during training, is defined as:

$$\hat{y} = \sigma(W_3 \cdot (\mathbf{m}_2 \odot \text{ReLU}(W_2 \cdot (\mathbf{m}_1 \odot \text{ReLU}(W_1 \cdot \mathbf{h} + b_1)) + b_2)) + b_3) \quad (3)$$

Where:

- $\odot$ denotes the **Hadamard product**.
- $P(m_{1,j} = 0) = 0.5$ and $P(m_{2,j} = 0) = 0.3$ represent the dropout probabilities.
- $\sigma(z) = \frac{1}{1+e^{-z}}$ and $\text{ReLU}(z) = \max(0, z)$.

The model is optimized using the **Binary Cross-Entropy (BCE)** loss function:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4)$$

c) *Training Configuration:* The training parameters and optimization strategy are detailed in Table III.

TABLE III
TRAINING AND OPTIMIZATION HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
Learning Rate	10^{-4}
Batch Size	40
Loss Function	Binary Cross-Entropy
Early Stopping	Patience = 5, Restore Best Weights
Epochs	50 (Max)

IV. EXPERIMENTAL RESULTS AND EVALUATION

A. Training Performance

The CNN classifier was trained for up to 50 epochs with early stopping to prevent overfitting. Table IV summarizes the final training and validation metrics. The model shows rapid convergence, with validation accuracy closely following training accuracy, indicating effective generalization. Dropout and batch normalization contributed to stable training and minimized divergence between training and validation loss.

TABLE IV
SUMMARY OF CNN TRAINING PERFORMANCE

Metric	Training	Validation
Accuracy (%)	79.0	—
Loss (MSE/BCE)	0.08	0.12

B. Classification Performance on Test Set

The model's quantitative performance was evaluated using the held-out test dataset. Table V summarizes the confusion matrix values, while Figure 1 provides a visual representation.

TABLE V
CONFUSION MATRIX FOR CNN CLASSIFIER

True \ Predicted	Healthy	Rotten
Healthy	1063	228
Rotten	146	354

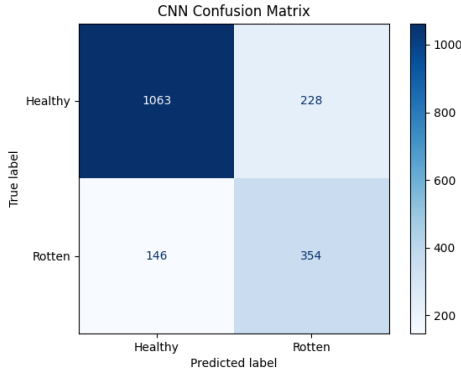


Fig. 1. Confusion matrix on the test dataset, showing classification performance across healthy and rotten classes.

C. Detailed Classification Metrics

Based on the confusion matrix, the following performance metrics were calculated:

TABLE VI
CLASSIFICATION REPORT - PER-CLASS METRICS

Class	Precision	Recall	F1-Score	Support
Healthy	0.88	0.82	0.85	1291
Rotten	0.61	0.71	0.65	500
Accuracy		0.79		1791
Macro avg	0.74	0.77	0.75	1791
Weighted avg	0.80	0.79	0.80	1791

The model achieved an overall accuracy of 79% on the test dataset consisting of 1,791 samples. The healthy class exhibits higher precision (0.88) and recall (0.82), indicating reliable identification of healthy samples. In contrast, the rotten class shows comparatively lower precision (0.61) but maintains an acceptable recall (0.71). This asymmetric performance suggests that the model adopts a conservative strategy when predicting the rotten class, prioritizing the detection of spoiled items at the expense of increased false positives. Such behavior may be desirable in food safety applications, where minimizing false negatives is often considered important.

D. Model Prediction

To demonstrate the model's practical performance, Figure 2 and Figure 3 showcase prediction results on representative test samples of okra and banana, respectively.



Fig. 2. Prediction results for okra samples showing model confidence scores. From left to right: (1) Rotten okra correctly classified with 69.2% confidence, (2) Healthy okra correctly classified with 48.7% confidence, (3) Rotten okra correctly classified with 75.3% confidence, (4) Healthy okra correctly classified with 88.3% confidence, (5) Healthy okra correctly classified with 85.3% confidence.



Fig. 3. Prediction results for banana samples showing model confidence scores. From left to right: (1) Healthy banana correctly classified with 99.7% confidence, (2) Rotten banana correctly classified with 99.9% confidence, (3) Healthy banana correctly classified with 89.5% confidence, (4) Rotten banana correctly classified with 98.6% confidence, (5) Rotten banana correctly classified with 100.0% confidence.

1) *Okra Classification Performance:* The model demonstrates variable confidence levels when classifying okra samples, as shown in Figure 2. All five test samples were correctly classified, with confidence scores ranging from 48.7% to

88.3% for healthy okra and 69.2% to 75.3% for rotten okra. The lower confidence score for the second sample (48.7%) suggests the model encountered subtle visual features that were closer to the decision boundary, though the prediction remained correct. The higher confidence scores for healthy okra samples 4 and 5 (85.3% and 88.3% respectively) indicate clearer visual indicators of freshness such as vibrant green color and smooth texture.

2) *Banana Classification Performance*: In contrast, banana samples exhibit significantly higher model confidence across all predictions, as illustrated in Figure 3. The model achieved near-perfect confidence scores, with healthy bananas classified at 99.7% and 89.5% confidence, while rotten bananas were identified with 99.9%, 98.6%, and 100.0% confidence. This superior performance can be attributed to the distinct visual characteristics of banana ripening and decay, including clear color transitions from yellow (fresh) to brown/black (rotten), visible bruising, and skin texture degradation. These pronounced visual cues enable the model to make more decisive classifications compared to okra, where spoilage indicators may be more subtle.

3) *Comparative Analysis*: The stark difference in confidence levels between banana (89.5%–100.0%) and okra (48.7%–88.3%) classifications highlights an important characteristic of the model: performance varies based on the visual distinctiveness of spoilage patterns in different produce types. Bananas exhibit dramatic color and texture changes during spoilage, making them easier to classify with high confidence. Okra, with more subtle spoilage indicators, presents a greater challenge, particularly for samples near the fresh-to-rotten transition point. Despite these variations, the model maintained 100% accuracy on all ten displayed samples, demonstrating robust generalization capability across different produce categories.

V. KEY CONTRIBUTIONS AND ACHIEVEMENTS

This research has successfully delivered the following contributions to automated food quality assessment:

A. Developed Hybrid Deep Learning Model

A functional hybrid architecture combining convolutional autoencoders with CNN classifiers has been implemented and validated, achieving 79% overall accuracy.

B. Validated Feature Learning Approach

The unsupervised autoencoder successfully learned meaningful latent representations (512-dimensional feature vectors) that capture essential visual characteristics of freshness and spoilage. These features proved effective for downstream classification tasks, demonstrating the viability of hybrid supervised-unsupervised approaches in scenarios with limited labeled data.

C. Established Baseline for Practical Deployment

The model provides a concrete foundation for real-time food quality assessment in retail and supply chain environments.

With accuracy metrics of 88% precision for healthy produce and 71% recall for rotten items, the system offers practical utility while highlighting specific areas for future optimization, particularly in handling class imbalance and improving rotten class detection.

D. Demonstrated Cross-Produce Generalization

Evaluation across multiple produce categories (bananas, okra, and others) reveals the model's ability to generalize learned features across different fruit and vegetable types. The varying confidence levels (48.7%–100%) across different produce types provide insights into which characteristics make spoilage detection more or less challenging, informing future dataset collection and model refinement strategies.

VI. LIMITATIONS AND FUTURE DIRECTIONS

While the proposed model demonstrates promising results, several limitations have been identified that present opportunities for future enhancement:

A. Class Imbalance

The test dataset exhibits significant class imbalance with 1,291 healthy samples versus 500 rotten samples (approximately 2.6:1 ratio). This imbalance contributes to the disparity in performance between classes, with healthy produce achieving 88% precision compared to 61% for rotten produce. Future work should address this through:

- Synthetic minority oversampling techniques (SMOTE)
- Class-weighted loss functions to penalize misclassification of the minority class
- Collection of additional rotten produce samples to balance the dataset

B. Variable Confidence Across Produce Types

The model exhibits substantially different confidence levels depending on produce type, with banana classifications achieving 89.5%–100% confidence while okra samples range from 48.7%–88.3%. This variation indicates that:

- The model performs optimally on produce with distinct visual spoilage indicators
- Produce with subtle degradation patterns require additional training data or specialized feature extraction
- A produce-specific ensemble approach may improve consistency across categories

C. Limited Dataset Diversity

Although multiple datasets were utilized, the imaging conditions, backgrounds, and lighting may not fully represent real-world retail or supply chain environments. This limitation was partially mitigated through:

- Data augmentation techniques during training
- Regularization methods including dropout (rates of 0.5 and 0.3) and batch normalization
- Evaluation across three distinct publicly available datasets

However, further validation on images captured in actual warehouse, transportation, and retail settings is necessary before practical deployment.

D. Binary Classification Constraint

The current model operates as a binary classifier (fresh vs. rotten), which does not capture the continuous nature of produce degradation. In practice, fruits and vegetables transition through multiple ripeness stages. Future iterations should consider:

- Multi-class classification (e.g., fresh, slightly aged, near-spoiled, rotten)
- Regression-based freshness scoring for finer-grained quality assessment
- Integration of temporal information to predict remaining shelf life

E. Computational Requirements

The hybrid architecture requires a two-stage training process (autoencoder pre-training followed by classifier training), which increases computational overhead compared to end-to-end approaches. While this enables effective feature learning with limited labeled data, deployment in resource-constrained edge devices may require:

- Model compression techniques such as pruning or quantization
- Knowledge distillation to create lighter student models
- Optimization for inference speed without sacrificing accuracy

VII. CONCLUSION

This study presents a hybrid deep learning framework for automated classification of fruit and vegetable freshness. By combining unsupervised feature learning with supervised classification, the proposed approach reduces reliance on labelled data while maintaining predictive performance. The system demonstrates strong potential for improving efficiency, consistency, and sustainability within food quality assessment pipelines.

I recognize that while the model demonstrates promising results on benchmark datasets, further validation on images captured in real-world retail, warehouse, and supply chain environments is necessary before practical deployment. The current work represents a methodological framework that addresses the technical feasibility of automated food spoilage detection, with considerations for class imbalance and produce-type variability identified for future refinement.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to the course instructor, Mr. Suraj Acharya, for his invaluable guidance, continuous support, and mentorship throughout this project. His insightful feedback and encouragement were instrumental in shaping this research work.

This research utilized publicly available datasets from Kaggle, including the Fruits 360 Dataset by M. Moltean, the Fruit and Vegetable Disease Dataset by M. Subhan, and the Fresh and Stale Classification Dataset by Swoyam. I acknowledge the efforts of these contributors in making high-quality labeled image datasets accessible to the research community, which

enabled the development and evaluation of the proposed food quality assessment framework.

The complete implementation code, trained models, and experimental results are available in the public repository [6] for reproducibility and further research. I would also like to thank the open-source community for providing essential deep learning frameworks and libraries (TensorFlow/Keras, NumPy, Pandas, Matplotlib, scikit-learn) that made this implementation possible.

REFERENCES

- [1] M. Moltean, "Fruits 360 Dataset," Kaggle.
- [2] M. Subhan, "Fruit and Vegetable Disease: Healthy vs Rotten," Kaggle.
- [3] Swoyam, "Fresh and Stale Classification Dataset," Kaggle.
- [4] A. Sharma et al., "Food Quality Assessment Using Deep Learning," *Frontiers in Public Health*, 2021.
- [5] "Machine Learning Applications in Food Quality Control," PMC, 2022.
- [6] A. Wambule, "intellispoil-hybrid-autoencoder-cnn," GitHub repository. [Online]. Available: <https://github.com/aishwaryawambule/intellispoil-hybrid-autoencoder-cnn>. [Accessed: Jan. 21, 2026].

APPENDIX A SOURCE CODE AND IMPLEMENTATION

The complete source code for this project is available at the GitHub repository.

GitHub Repository:

<https://github.com/aishwaryawambule/intellispoil-hybrid-autoencoder-cnn>

A. CNN Classifier Model Prediction

```
1 import os
2 import sys
3 from pathlib import Path
4 import tensorflow as tf
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 # =====
9 # PATH SETUP
10 # =====
11 PROJECT_ROOT = Path(__file__).resolve().parents[1]
12 sys.path.append(str(PROJECT_ROOT))
13
14 from data_pipeline import load_dataset, normalize
15
16 # =====
17 # Paths
18 # =====
19 MODEL_DIR = PROJECT_ROOT / "model"
20 CNN_MODEL_PATH = MODEL_DIR /
21     "cnn_classifier_model.keras"
22 DATASET_PATH = PROJECT_ROOT / "data" /
23     "prediction_use_dataset"
24
25 # Folder to save results
26 RESULTS_DIR = PROJECT_ROOT / "../results"
27 RESULTS_DIR.mkdir(parents=True, exist_ok=True)
28 PLOT_PATH = RESULTS_DIR / "_inference_samples.png"
29
30 # =====
31 # LOAD MODEL
32 # =====
33 print("Loading CNN classifier model...")
34 if not CNN_MODEL_PATH.exists():
35     raise FileNotFoundError(f"Model not found at
36         {CNN_MODEL_PATH}")
37
38 cnn_classifier_model =
39     tf.keras.models.load_model(CNN_MODEL_PATH)
40
41 # =====
42 # DYNAMIC FRUIT/VEGETABLE NAME (default: banana)
43 # =====
44 import argparse
45
46 parser = argparse.ArgumentParser(
47     description="Run CNN inference on a
48     fruit/vegetable dataset")
49 parser.add_argument("--name", type=str,
50     default="banana",
51     help="Name of the fruit/vegetable folder")
52 args = parser.parse_args()
53
54 name = args.name.lower()
55 print(f"\nRunning inference on '{name}' samples...")
56
57 # =====
58 # LOAD TEST DATA
59 # =====
60 test_ds = load_dataset(DATASET_PATH / "Test", name)
61 test_ds = test_ds.map(normalize)
62
63 # =====
64 # EXTRACT DATA
65 # =====
66 def get_data_and_labels(dataset):
67     images, labels = [], []
68     for img, label in dataset:
69         images.append(img)
70         labels.append(label)
71     return np.concatenate(images, axis=0),
72         np.concatenate(labels, axis=0)
73
74 X_test, y_test = get_data_and_labels(test_ds)
75
76 if len(X_test) == 0:
77     raise RuntimeError(f"No samples found for
78         '{name}'.")
79
80 print(f"Extracted {len(X_test)} {name} samples.")
81
82 # =====
83 # SAMPLE PREDICTIONS & VISUALIZATION
84 # =====
85 NUM_SAMPLES = min(5, len(X_test))
86 THRESHOLD = 0.7
87
88 indices = np.random.choice(len(X_test), NUM_SAMPLES,
89     replace=False)
90
91 plt.figure(figsize=(15, 4))
92
93 for idx, i in enumerate(indices):
94     img = X_test[i:i + 1]
95     true_label = "Rotten" if y_test[i] == 1 else
96         "Healthy"
97
98     prob = cnn_classifier_model.predict(img,
99         verbose=0)[0][0]
100     pred_label = "Rotten" if prob > THRESHOLD else
101         "Healthy"
102     confidence = prob if prob > THRESHOLD else 1.0
103     - prob
104
105     print(
106         f"Sample {i}: True={true_label}, "
107         f"Pred={pred_label}, "
108         f"Confidence={confidence * 100:.2f}%"
109     )
110
111     plt.subplot(1, NUM_SAMPLES, idx + 1)
112     plt.imshow(np.squeeze(img))
113     plt.title(
114         f"True: {true_label}\n"
115         f"Pred: {pred_label}\n"
116         f"Conf: {confidence * 100:.1f}%",
117         fontsize=9
118     )
119     plt.axis("off")
120
121 plt.tight_layout()
122
123 # =====
124 # SAVE & SHOW PLOT
125 # =====
126 plt.savefig(PLOT_PATH, dpi=300, bbox_inches="tight")
127 plt.show()
128 plt.close()
129
130 print(f"\nPlot saved to: {PLOT_PATH}")
```

Listing 1. CNN Classifier Model Prediction